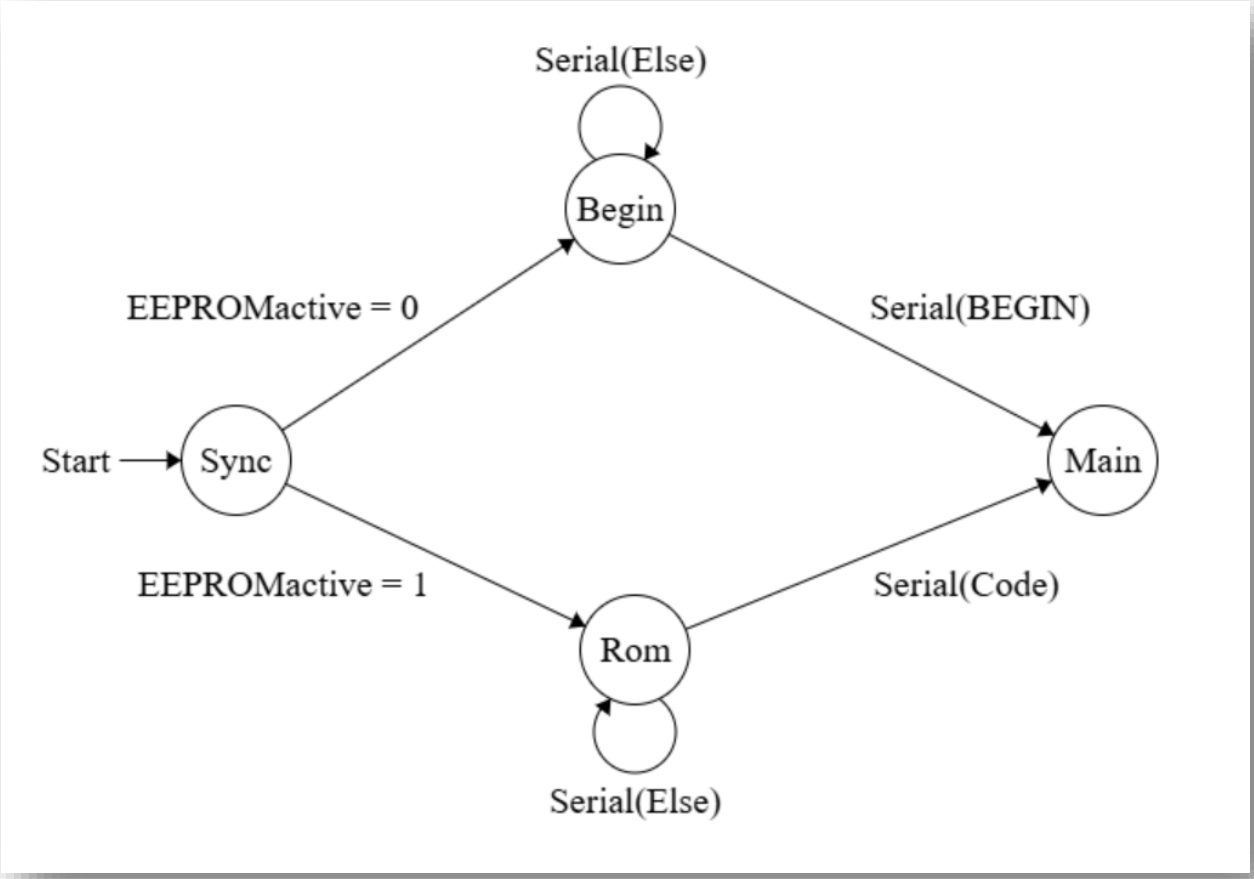
24COA202 Coursework

F418497

Semester 1

1 FSMs



Sync state

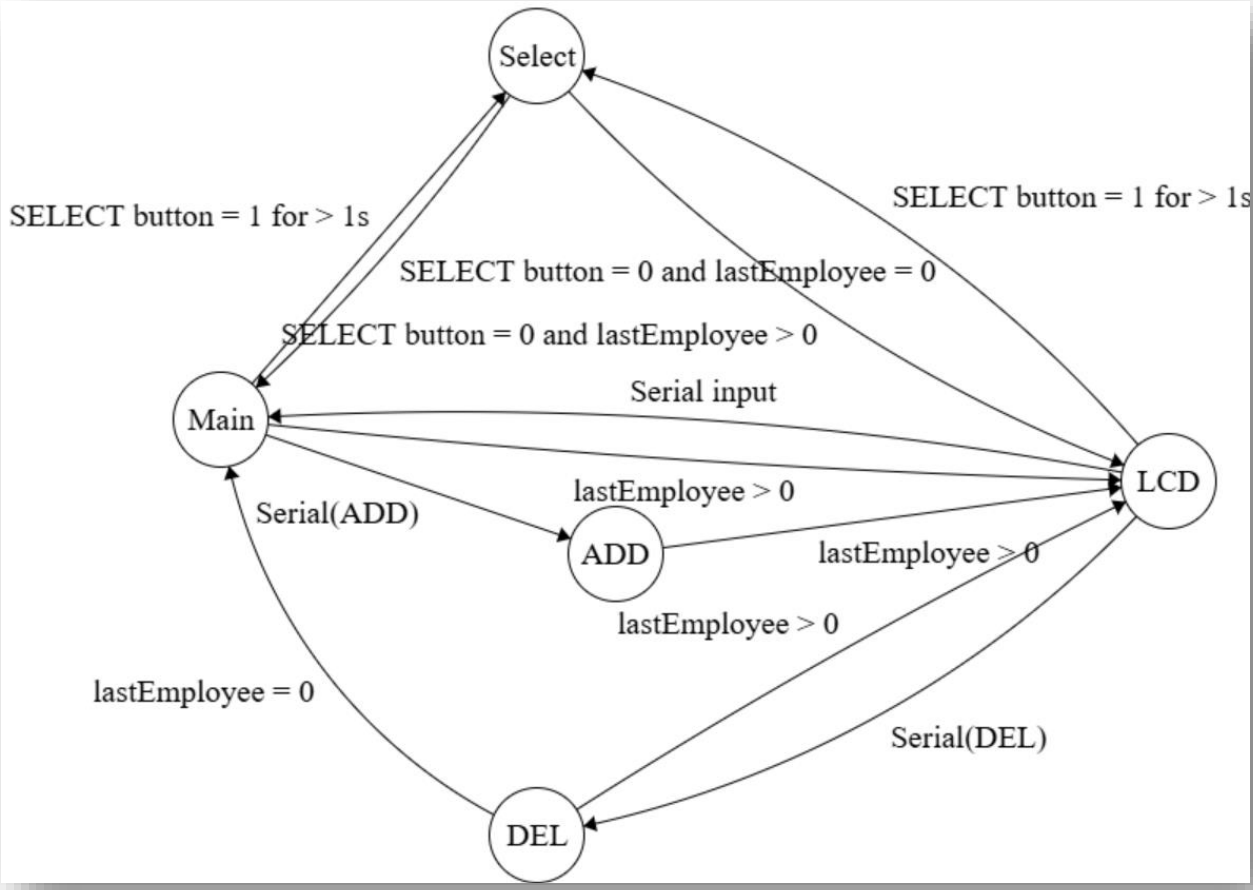
Turns background lcd yellow, reads the ROM code and reads the EEPROM active bit

Begin state

Sends an R every 2 seconds to the Serial Monitor, constantly polls for serial inputs and outputs an error if the input is not 'BEGIN'. If the input is 'BEGIN', 'BASIC' is outputted to the Serial Monitor.

Rom state

Sends an R every 2 seconds to the Serial Monitor, constantly polls for serial inputs and outputs an error if the input is not the ROM code. If the input is 'BEGIN', 'UDCHARS,FREERAM,EEPROM,SCROLL' is outputted to the Serial Monitor.



Main state

Turns the LCD backlight to white and constantly polls for serial inputs. If a protocol command is inputted, it is processed to ensure that it is in the correct format and then changes the employee data as necessary. The employee ID number is also sorted using bubble sort so that each employee is displayed in a numerical order. After every command, the LCD screen is also cleared to prevent characters remaining on the screen from previous states.

Select state

Turns LCD backlight to purple and prints the student ID number at the top of the screen and the free memory available in SRAM at the bottom. While it does this, it constantly polls the serial input and processes the commands.

ADD state

Increments the last employee count by 1 and saves all employee information entered. If all employee slots are full, an error is returned unless another employee is deleted and there is enough space for a new employee.

DEL state

Decrements the last employee count by 1 and moves all employee information entered after the deleted account backwards by 1 so that the next employee added is added from the end. Also adjusts the current employee viewed on the LCD so that if they are deleted, the previous employee is instead viewed unless it is the first employee. Sends an error if the command is formatted incorrectly

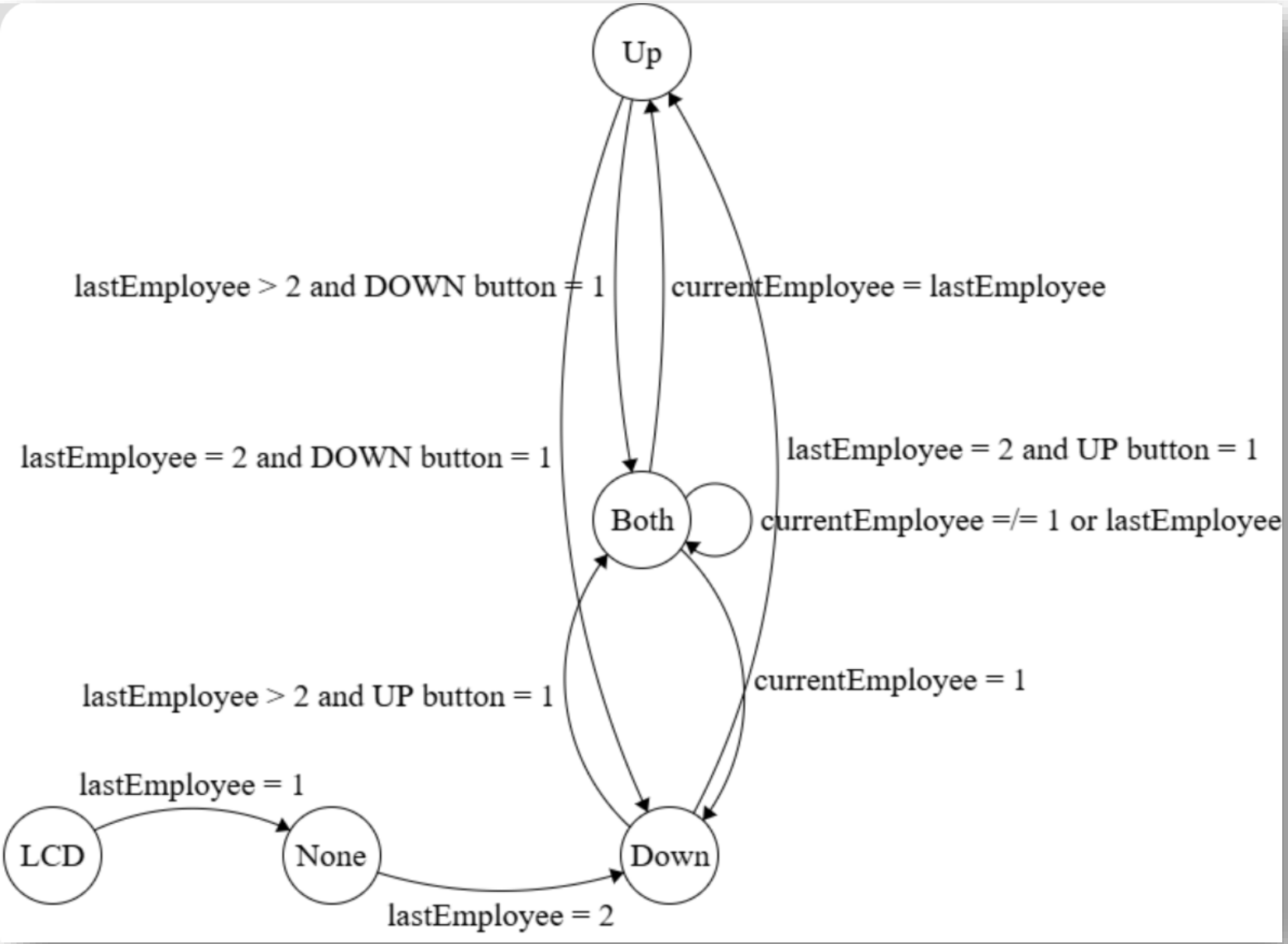
LCD state

Constantly updates the LCD screen depending on the current employee chosen. If the current employee has a pension, the screen has a green background, otherwise it is red. Displays, the employees' job grade, pension status, salary, ID and job title on the LCD. Empty spaces in the LCD are also replaced by an empty char to override previous letters displayed.

Transitions

- EEPROMactive = 1 (when the EEPROM is active)
- EEPROMactive = 0 (when the EEPROM is deactivated)
- Serial(Else) (when the serial input is different from the desired message)
- Serial(Begin) (when the serial input is 'BEGIN')
- Serial(Code) (when the serial input is the code stored in the EEPROM)
- Select Button = 1 for > 1s (when the select button is held for more than 1 second)
- Select Button = 0 and lastEmployee = 0 (when the select button is unpressed and there are no employees)
- Select Button = 0 and lastEmployee > 0 (when the select button is pressed and there are employees)
- Serial input (any serial input)
- lastEmployee > 0 (when there are employees)
- lastEmployee = 0 (when there aren't employees)
- Serial(ADD) (when the serial input is an ADD command)
- Serial(DEL) (when the serial input is a DEL command)

Sub-FSMs for LCD display



None state

When there is only one employee, no arrows are displayed on the LCD.

Down state

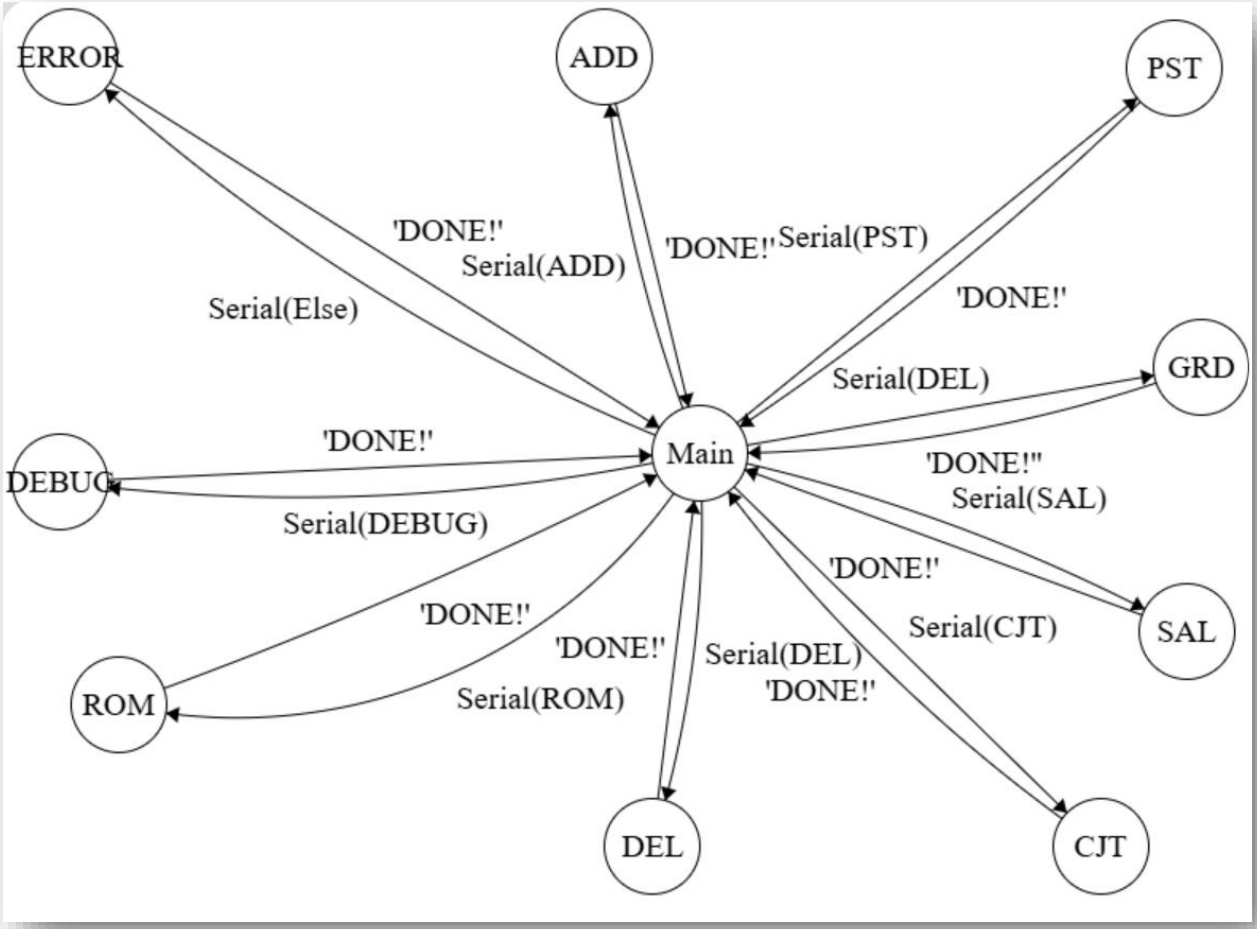
When there is more than one employee, the first state of the LCD display will always be the down state and will have an arrow pointing upwards to change to other employees

Up state

When there is more than one employee and the current employee shown is the last available employee in the database, an arrow pointing downwards is used to change to previous employees.

Both state

When there are more than 2 employees and the current employee is neither the first employee or last employee, then both arrows are displayed to indicate that either the next employee or previous employee can be selected.



PST state

Changes the pension status of an employee. If the employee doesn't exist or the command is formatted incorrectly, an error is sent. If the pension status is unchanged, an error is also sent. If the salary of an employee is £0, sends an error.

GRD state

Changes the job grade of an employee. If the employee doesn't exist or the command is formatted incorrectly, an error is sent. If the job grade is the same or below the previous value, it is unchanged, and an error is sent.

SAL state

Changes the salary of an employee. If the employee doesn't exist or the command is formatted incorrectly, an error is sent. If the updated money is £0, sends an error.

CJT state

Changes the job title of an employee. If the employee doesn't exist or the command is formatted incorrectly, an error is sent. If the job title is unchanged, an error is sent.

ROM state

Stores a 1-10 alphabetic code, which writes to the EEPROM to store. Enables the use of EEPROM. If the same command is used with the same code, the EEPROM is disabled. Used to change the code needed to enter main phase of the system.

Debug state

Prints the entire entered command string and certain variables used for debugging,

Error state

If the first 3 letters of the command are entered incorrectly, sends an error.

2 Data structures

An array of structs is used to store the employee information in the code

An Enum is used for each input command to convert the String into an integer so that it can be used in a switch-case

A String was used for the ID to easily display on the LCD. However, this comes at the large cost of heap fragmentation from the use of Strings. Instead, an unsigned long could have been used to store digits.

A char was used for the job grade to easily display a single character on the LCD and easily convert to an integer for operations

A String was used for the job title to easily display on the LCD and use the substring function to easily implement the scroll extension. However, this also comes at the cost of heap memory fragmentation. Instead, a char array of size 18 could've been used to store these characters.

A float was used for the salary to easily round the currency to its nearest 2dp value.

A String was also used for the pension to store the status and easily display this on the LCD. However, this brings the problem of heap fragmentation from the other String classes and could be replaced by a Boolean variable to store the status.

The constants were error messages stored as Strings in the flash memory using the F() macro.

sortEmployees – uses bubble sort to rearrange the employee IDs into numerical order

addInput – adds new employee data to the array of structs

pstInput – changes selected employee pension status

grdInput – changes selected employee job grade

salInput – changes selected employee salary

cjtInput – changes selected employee job title

delInput – deleted selected employee from the array of structs

3 Debugging

```
case DEBUG:
  if (input.substring(0, 6) == "DEBUG:") {
    Serial.println(input);
    // Serial.println(EEPROMactive);
    // Serial.println(romCode);
    // Serial.println(lastEmployee);
    // Serial.println(currentEmployee);
```

Used the debug function to test the value of variables at different points in the code
E.g. EEPROMactive and romCode were useful in testing whether the EEPROM was meant to be active and what the saved code was
lastEmployee and currentEmployee were also useful in testing the functionality of arrows and scrolling with the LCD

```
} else if (q <= employees[currentEmployee].jobTitle.length() - 7) { // otherwise scroll at 1 character every 0.5s
  if (millis() - time >= 500) {
    lcd.setCursor(9, 1);
    lcd.print(employees[currentEmployee].jobTitle.substring(q, employees[currentEmployee].jobTitle.length()));
    time = millis();
    q++;
  }
} else {
  q = 0; // reset the scroll when the job title has reached its end
}
```

Debugging used where a for loop couldn't be used as it skips millis() function and can't delay while looping.
Solution was to create if statements in the main loop to test the conditions and increment each time until the condition isn't met, in which case it is reset.

```
PS C:\Users\natnk> python nameoftestfile.py COM6
connecting to port COM6
waiting for sync
Sync
b'UDCHARS,FREERAM,EEPROM,SCROLL\n'
Write: b'ADD-6359870-6-Software_Engineer'
Write: b'ADD-3356872-4-Guard'
Write: b'ADD-1425466-8-HR_Manager'
Write: b'SAL-6359870-45000'
Write: b'PST-6359870-PEN'
Read:  b"ERROR: Employee pension status hasn't changed.\r\n"
Write: b'PST-1425466-PEN'
Write: b'GRD-1425466-50'
Read:  b"ERROR: Employee pension status hasn't changed.\r\n"
Write: b'PST-6359870-PEN'
Write: b'PST-6359870-1'
Read:  b'ERROR: Command entered is the incorrect length.\r\n'
Write: b'GRD-3499234-6'
Write: b'SAL-1425466-71250'
Read:  b"ERROR: Employee pension status hasn't changed.\r\n"
Write: b'PST-1425466-PEN'
Write: b'CJT-3356872-Security-Guard'
Read:  b'ERROR: Command entered is the incorrect length.\r\n'
Write: b'SAL-3356872-31300'
Write: b'ADD-23577172-5-Care_Taker'
Read:  b'ERROR: ID does not match an employee ID.\r\n'
Write: b'ADD-2357712-Care_Taker'
Write: b'ADD-2357712-5-Care@Taker'
Write: b'ADD-2357712-5-CT'
Read:  b"ERROR: Employee pension status hasn't changed.\r\n"
Write: b'ADD-2357712-5-'
Write: b'ADD-2357712-5-Care_Taker'
Read:  b'ERROR: Job title entered contained characters without letters, numbers, .s or _s\r\n'
Write: b'ADD-3356872-7-Swim_Instructor'
Write: b'ADD9872334-7-Swim_Instructor'
Write: b'ADD9872334-7- Swim_Instructor'
Read:  b'ERROR: Command entered is the incorrect length.\r\n'
```

Use of python script to automate testing in Arduino.

Each error is correct but responds too late compared to the write, which is not a problem when in proper use as typing in commands won't be as fast as the writing function for this script.

4 Reflection

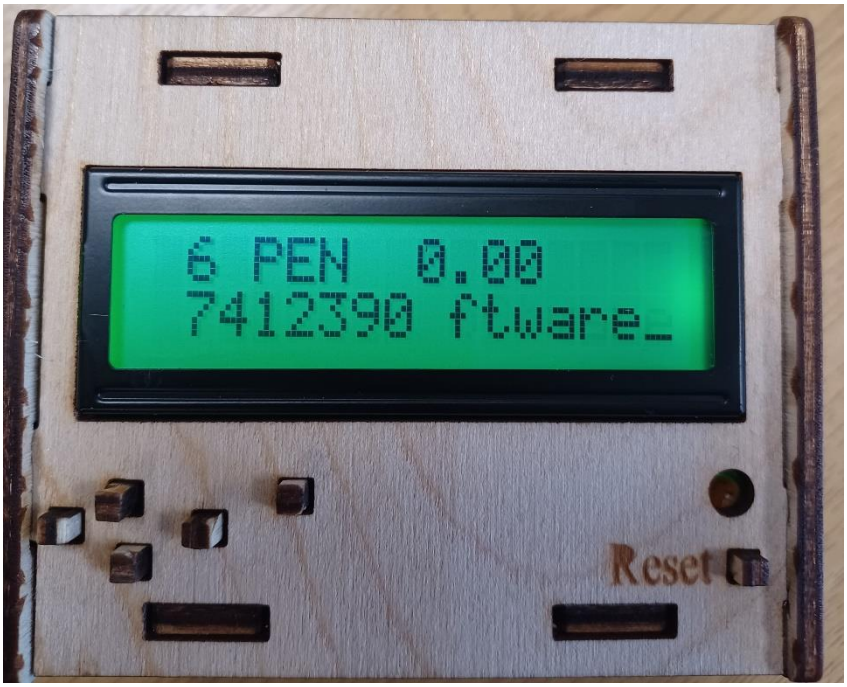
The code works well in fulfilling the specification accurately. However, the code is not very efficient as it contains multiple Strings in the array of structs, which will result in heap fragmentation in the memory. This means that much less employee data can be stored than an optimised use of structs using char arrays and Boolean variables to store data. This results in a maximum of 22 employees' data being available to store, which is still more than enough for a small team. Furthermore, the sort method (to sort the employee IDs numerically) used is bubble sort, which is notoriously sub-optimal. However, in this code, sorting does not need to be too quick as a maximum of only 21 other employees will have to be sorted. In this system, error messages are specific to the type of error made as to update the error made by the user and allow them to fix it when typing in the command. As well as this, the system successfully determines errors in any way that a person may enter a command incorrectly. This means that data interpreted by the commands are guaranteed to be correctly saved corresponding to each employee and will be consistent. Additionally, the LCD display contains fail safes so that in the result of characters remaining on the screen from previous states, they will reset the LCD display with every button press or command input. However, this is unlikely to happen in the first place as empty characters are printed where characters are not needed to display. Overall, I would say that the code is simple but completes the specification as necessary and would be easy for another programmer to make changes and understand.

5 UDCHARS

```
byte upArrow[8] = { // create up arrow character to display on LCD
                    0b00100,
                    0b01110,
                    0b10101,
                    0b00100,
                    0b00100,
                    0b00100,
                    0b00100,
                    0b00100
};
lcd.createChar(1, upArrow);
byte downArrow[8] = { // create down arrow character to display on LCD
                      0b00100,
                      0b00100,
                      0b00100,
                      0b00100,
                      0b00100,
                      0b10101,
                      0b01110,
                      0b00100
};
lcd.createChar(0, downArrow);
```

```
if (lastEmployee == 1) {  // when there is only one employees don't display any arrows
  lcd.setCursor(0, 0);
  lcd.print(' ');
  lcd.setCursor(0, 1);
  lcd.print(' ');
} else if (currentEmployee == 0) {  // when there is more than one employee but it's at
  lcd.setCursor(0, 0);
  lcd.write((uint8_t)1);
  lcd.setCursor(0, 1);
  lcd.print(' ');
  if (buttons & BUTTON_UP) {  // press the up button to scroll to the next employee
    ++currentEmployee;
    lcd.clear();
  }
} else if (currentEmployee == lastEmployee - 1) {  // when there is more than one emplo
  lcd.setCursor(0, 0);
  lcd.print(' ');
  lcd.setCursor(0, 1);
  lcd.write((uint8_t)0);
  if (buttons & BUTTON_DOWN) {  // press the down button to scroll to the previous empl
    --currentEmployee;
    lcd.clear();
  }
} else {
  lcd.setCursor(0, 0);  // when there are more than one employees but not at the end or
  lcd.write((uint8_t)1);
  lcd.setCursor(0, 1);
  lcd.write((uint8_t)0);
```

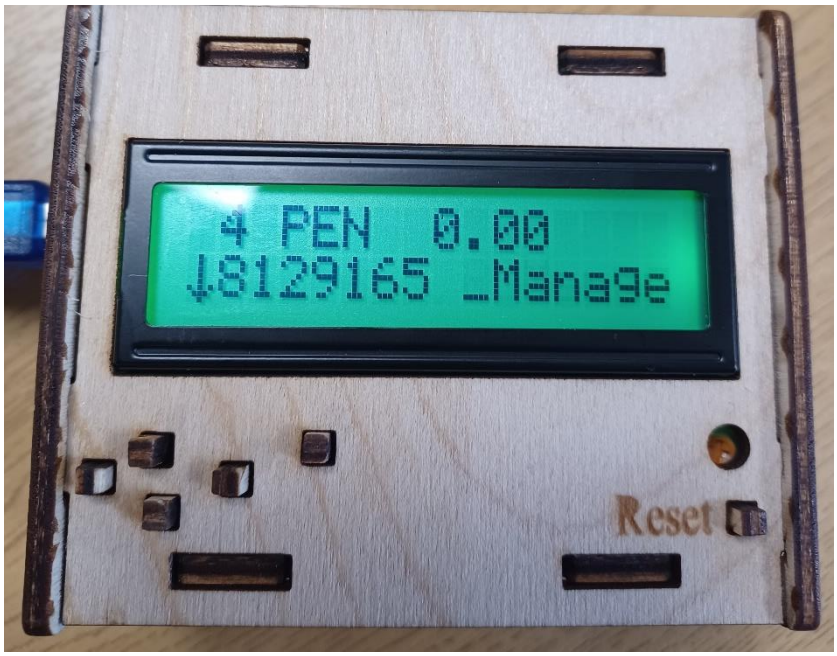
For this extension, I created a byte array for the up and down arrow as well as the createChar function from the LCD to create a single character on the LCD screen. This didn't change the FSM at all as the only difference was using arrows rather than alphabetic characters.



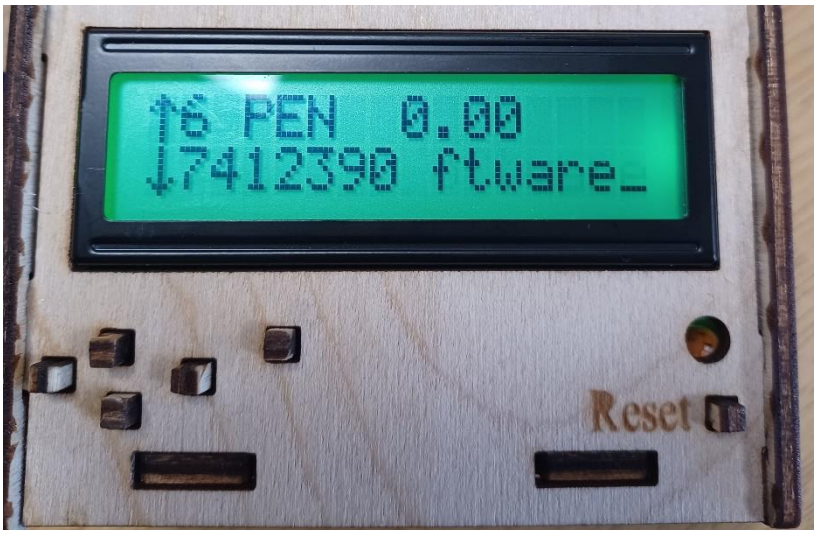
Here, no arrows are displayed as there is only one employee available



Here, the up arrow is displayed as there is more than one employee that the user can access, but the user is at the bottom of the list of employees.

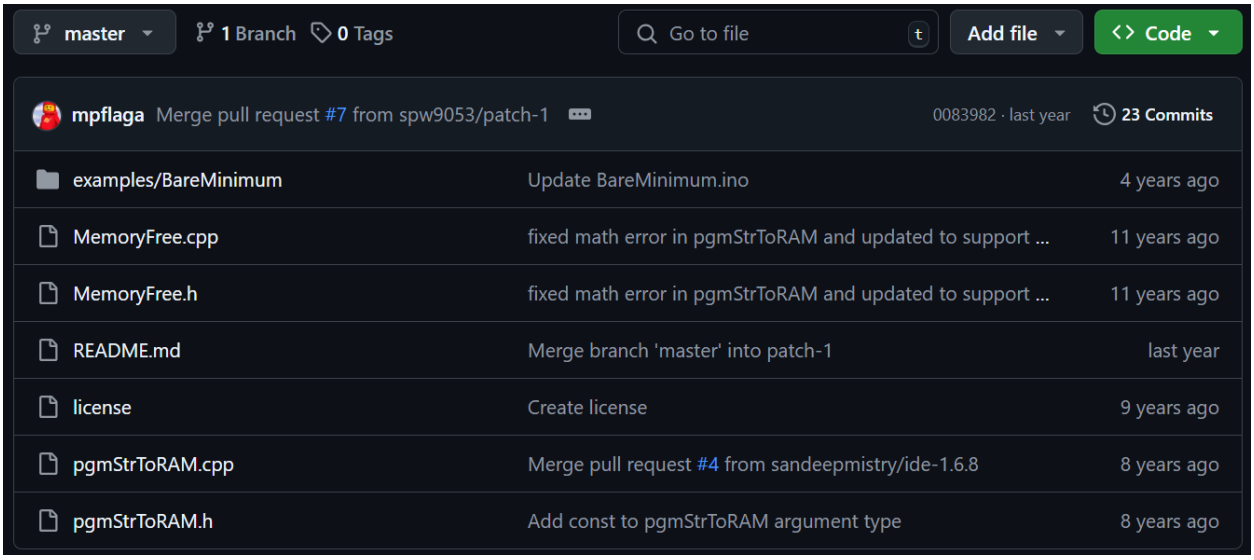


Here, the down arrow is displayed as there is more than one employee that the user can access, but the user is already at the top of the list of employees.



Here, both arrows are displayed as there is more than two employees that the user can access, and the user is neither at the top or bottom of the list of employees.

6 FREERAM




```
#include <MemoryFree.h>
```

```
  lcd.setCursor(0, 1);
  lcd.print(F("Free SRAM:"));
  lcd.setCursor(11, 1);
  lcd.print(freeMemory());
}
```

I downloaded the MemoryFree.h library from GitHub, put it in the Arduino library folder and used the freeMemory() function to print on the LCD the free RAM available when SELECT is held for more than 1 second. This changed the FSM as when the machine is in the SELECT state, it displays the free SRAM available as well as the student ID.



Here, the free SRAM is displayed as the select button is pressed

7 EEPROM

```
void romInput(String input) {
  if (input.length() < 5 || input.length() > 14) {  // if command entered is the incorrect length, send an error
    Serial.println(F("ERROR: Command entered is the incorrect length."));
    return;
  } else if (input.charAt(3) != '-') {  // inbetween characters are not dashes, send an error
    Serial.println(F("ERROR: Command has dashes in incorrect placement."));
    return;
  } else {
    memset(charArray, 0, sizeof(charArray));        // erases array data
    for (int i = 0; i < input.length() - 4; i++) {  //ensure that all digits in the command are alphabetic
      if (!isalpha(input.substring(4, input.length())[i])) {
        Serial.println(F("ERROR: ROM code entered contained digits other than alphanumeric characters."));
        return;
      }
    }
    input.substring(4, input.length()).toCharArray(charArray, sizeof(charArray));  // convert the command to a char array
    for (int i = 0; i < sizeof(charArray); i++) {
      readArray[i] = EEPROM.read(i);  // read all values from the EEPROM to an array
    }
```

```
    romCode = String(readArray);                        // convert the array to a string
    if (input.substring(4, input.length()) == romCode) {  // if the EEPROM string is the same as the command, deactivate EEPROM
      romCode = ' ';
      EEPROMactive = false;
      EEPROM.write(19, EEPROMactive);
      return;
    } else {  // otherwise, write the new command into EEPROM and activate EEPROM
      for (int i = 0; i < sizeof(charArray); i++) {
        EEPROM.write(i, charArray[i]);
      }
      EEPROMactive = true;
      EEPROM.write(19, EEPROMactive);
    }
  }
}
```

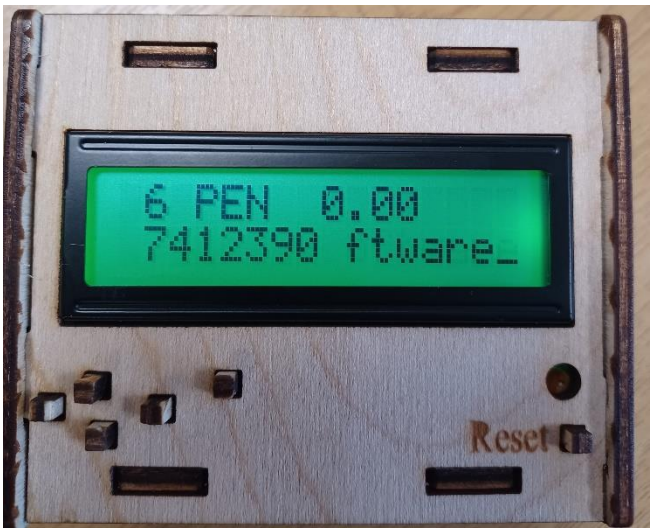
```
for (int i = 0; i < sizeof(readArray); i++) {  // read the ROM value stored in the EEPROM and convert to a string
  readArray[i] = EEPROM.read(i);
}
romCode = String(readArray);
EEPROMactive = EEPROM.read(19);  // read to see if EEPROM is active
while (syncPhase == true) {
  String input = Serial.readStringUntil('\r\n');  // during sync phase, constantly poll to read the input
  if (millis() - time >= 2000) {                  // print R to serial monitor every 2 seconds
    Serial.print('R');
    time = millis();
  }
  if (EEPROMactive == false) {  // if EEPROM is deactivated, start sync phase as normal
    beginInput(input);
  } else {  // otherwise, enter code to start main phase
    if (input == romCode) {
      Serial.print(F("UDCHARS,FREERAM,EEPROM,SCROLL"));  // print UDCHARS,FREERAM,EEPROM,SCROLL to the serial monitor
      Serial.print('\n');
      lcd.setBacklight(7);
      syncPhase = false;  // begin main sequence
      return;
    } else if (input != NULL) {
      Serial.println('R');  // error if input command is not the ROM code
      Serial.println(F("Error: String entered was not recognised."));
    }
```

For this extension, I completed the ROM function to write a code into the Serial Monitor and save this in a char array in the EEPROM as well as the EEPROM active variable to detect when this function is available. Typing in the command again will deactivate the EEPROM and ensure that a code isn't necessary to enter the main phase. However, I could not save the employee data to the EEPROM when it is activated as the excessive use of strings caused heap fragmentation and would cause errors when saved. This ensures that the FSM has two routes for the Synchronisation phase as the code must be inputted when EEPROM is active and BEGIN must be inputted when it is not. This is determined by the status of the EEPROM being read on setup and then using if statements to make decisions based on the status.

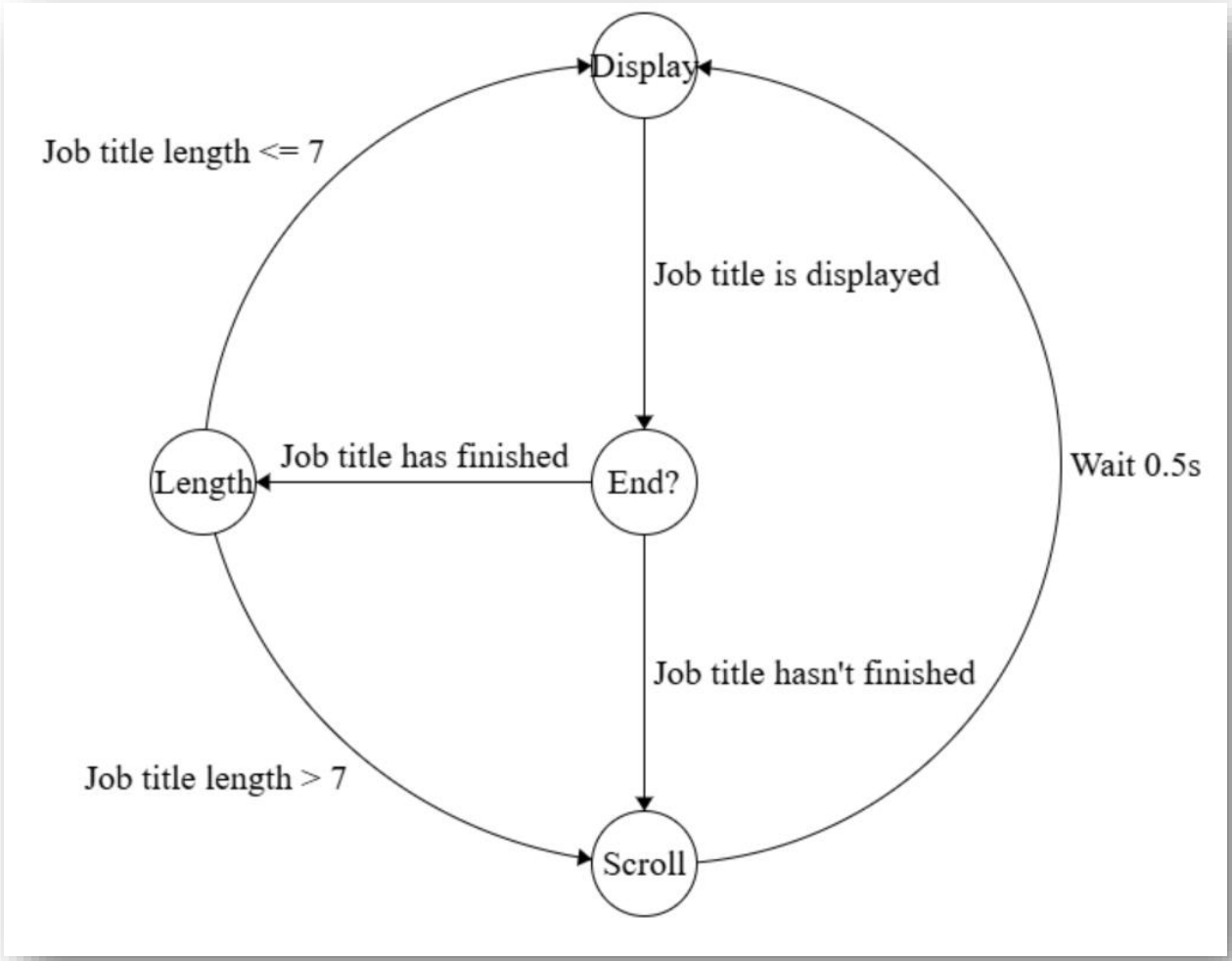
8 SCROLL

```
if (employees[currentEmployee].jobTitle.length() <= 7) {  // if the job title is smaller than 7 characters, don't scroll
  lcd.print(employees[currentEmployee].jobTitle + ' ' + ' ');
} else if (q <= employees[currentEmployee].jobTitle.length() - 7) {  // otherwise scroll at 1 character every 0.5s
  if (millis() - time >= 500) {
    lcd.setCursor(9, 1);
    lcd.print(employees[currentEmployee].jobTitle.substring(q, employees[currentEmployee].jobTitle.length()));
    time = millis();
    q++;
  }
} else {
  q = 0;  // reset the scroll when the job title has reached its end
}
```

When the job title is greater than the length of the screen, the job title will scroll from left to right at 1 character per 0.5 seconds until it reaches the end, where it then loops back to the start. This doesn't change the FSM as the states do not change, but only the LCD display



This image also shows the scroll function for the job title 'Software_Engineer' as each character is scrolled every 0.5s.



Length state

Checks the length of the job title

Scroll state

Moves one character to the right on the job title string

Display state

Displays the current job title string on the LCD screen

End state

Checks whether the job title has reached the end of its length. This means that the job title has scrolled enough to show the end of the title and would still have 7 characters more to scroll but this would cause blank spaces to also be scrolled.